Doctor Appointment & Patient Portal

Software Requirements Specification (SRS)

Course: CSAI 203 – Introduction to Software Engineering

Team Number: [null]

Team Members

Name	University ID
Mohamed Wafa	202401579
Karim Ezz	202401564
Ibrahim Ahmed	202402246
Assem Aldurini	202400587

Team Representative

Name: [Mohamed Wafa]

University ID: [202401579]

Email: [s-mohamed.wafa@zewailcity.edu.eg]

DOCTOR APPOINTMENT & PATIENT PORTAL

Software Requirements Specification (SRS)

Course: CSAI 203 – Introduction to Software Engineering

Semester: Fall 2026

Project Phase: Phase 2

Team ID: [Insert Team ID]

Team Members: [Insert Names and Student IDs]

Team Representative: [Insert Name and Student ID]

Submission Date: [Insert Date]

Table of Contents

1. Introduction
2. Overall Description
3. Specific Requirements
4. Appendices

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements of the Doctor Appointment & Patient Portal. It describes the system's functionality, users, constraints, interfaces, and quality requirements.

The document serves as a reference for the development and testing teams and establishes what the proposed system is expected to do.

1.2 Scope

The Doctor Appointment & Patient Portal is a web-based platform that connects patients with doctors and simplifies appointment management.

The system will allow patients to:

- Register and manage their accounts.
- Search for doctors by name or specialization.
- View doctor profiles and available appointment slots.
- Book, cancel, and reschedule appointments.
- View appointment history, medical history, and prescriptions.

Doctors will be able to manage their availability, view appointments, access relevant patient information, and issue prescriptions.

Administrators will manage user accounts and monitor registered users.

Out of scope: Emergency services, video consultations, online payments, automated diagnosis, external hospital-system integration, and pharmacy delivery.

1.3 Definitions and Abbreviations

Term	Definition
SRS	Software Requirements Specification
Patient	User who searches for doctors and books appointments
Doctor	Medical professional who manages appointments and prescriptions
Administrator	User responsible for managing platform accounts
Appointment	Scheduled consultation between a patient and doctor
Availability Slot	Time period offered by a doctor for booking
Prescription	Medication instructions issued by a doctor
FR	Functional Requirement
NFR	Non-Functional Requirement
MVC	Model-View-Controller architectural pattern
EHR	Electronic Health Record

1.4 References

- CSAI 203 – Introduction to Software Engineering, Fall 2026 Course Project Description.
- Course-provided Phase 2 SRS outline and requirements.
- IEEE-style SRS and use-case documentation conventions.

1.5 Document Overview

Section 2 describes the system and its operating environment. Section 3 presents the functional requirements, use-case model, conceptual class model, non-functional requirements, and interfaces. Section 4 contains the data dictionary and glossary.

2. Overall Description

2.1 Product Perspective

The system is a standalone web-based application using a client-server architecture. Users access it through a web browser, while the application server processes requests and communicates with a database.

The platform supports three main authenticated roles: Patient, Doctor, and Administrator. Visitors may also browse publicly available doctor information and register.

2.2 Product Functions

The main system functions are:

1. User registration, login, and profile management.
2. Doctor browsing, searching, and profile viewing.
3. Appointment availability management and booking.
4. Appointment cancellation and rescheduling.
5. Doctor schedule and appointment management.
6. Patient medical-history access.
7. Prescription creation and viewing.
8. Administrative user-account management.

2.3 User Classes and Characteristics

Patients: Search for doctors, manage appointments, and access their own medical information. They may have varying levels of technical experience.

Doctors: Manage availability, review assigned appointments, access permitted patient information, and issue prescriptions.

Administrators: Manage user accounts and oversee platform information.

Visitors: Browse public doctor information and register for an account.

2.4 Software Stakeholders

Stakeholder	Interest
Patients	Convenient access to doctors and appointment management
Doctors	Efficient schedule and prescription management
Administrators	Account management and platform oversight
Development Team	Design, implementation, and testing
Course Instructor / TA	Evaluation against project requirements

2.5 Operating Environment

- Client devices: Computers, laptops, tablets, and smartphones.
- Operating systems: Windows, macOS, Linux, Android, and iOS.
- Client software: Modern web browser.
- Server: Application server capable of processing user requests.
- Database: Database management system for storing application information.
- Network: Internet connection required for normal operation.

The exact technologies will be selected by the development team.

2.6 Design and Implementation Constraints

1. The system must support patient, doctor, and administrator roles.
2. Access must be restricted according to user permissions.
3. Appointment booking must prevent duplicate reservations.
4. Patient information must be protected from unauthorized access.
5. Data relationships must remain consistent.
6. The implementation must follow the MVC architectural pattern, as required by the course.
7. The selected technologies must be compatible with the team's development environment.

2.7 User Documentation

The system should provide basic instructions for registration, login, doctor searching, appointment booking, schedule management, prescription creation, and administrative account management. Clear validation and confirmation messages should also be provided.

2.8 Assumptions and Dependencies

- Users have access to an internet-connected device.
- Doctors maintain accurate availability information.
- Users provide valid registration information.
- Each account has an assigned role.
- The application server and database are operational.
- Users are responsible for protecting their login credentials.
- External payment and hospital systems are not required.

3. Specific Requirements

3.1 Functional Requirements

The following requirements define the services the system must provide.

FR-01: Patient Registration

The system shall allow visitors to register as patients by submitting their name, email, password, and required contact information. It shall validate the submitted information and reject incomplete or invalid registrations. Email addresses must be unique. Successful registration shall create a patient account and display confirmation.

FR-02: User Login

The system shall allow registered patients, doctors, and administrators to authenticate using their credentials. After successful authentication, users shall be directed to the interface associated with their role. Invalid credentials shall result in an error message without exposing sensitive account information.

FR-03: User Logout

The system shall allow authenticated users to log out. The active session shall be terminated, and protected pages shall no longer be accessible without authentication.

FR-04: Patient Profile Management

The system shall allow patients to view and update permitted profile information, such as their name, phone number, and address. The system shall validate changes and prevent patients from modifying protected account fields or another user's information.

FR-05: Browse Doctors

The system shall allow visitors and patients to browse available doctor profiles. Each public profile shall display the doctor's name, specialization, professional description, and relevant availability information.

FR-06: Search Doctors

The system shall allow users to search for doctors by name or medical specialization. Matching profiles shall be displayed. If no results match the search, the system shall display an appropriate message.

FR-07: View Doctor Details

The system shall allow users to select a doctor and view their detailed public profile, including specialization, professional description, and available appointment information. Private information shall not be exposed.

FR-08: View Available Slots

The system shall allow patients to view a selected doctor's available appointment dates and times. Booked or otherwise unavailable slots shall not be presented as available for reservation.

FR-09: Book Appointment

The system shall allow authenticated patients to reserve an available appointment slot. Before confirming a booking, the system shall verify that the slot is still available. A successful booking shall be associated with the selected patient, doctor, and time slot. If the slot has already been booked, the system shall reject the request and ask the patient to select another slot.

FR-10: View Patient Appointments

The system shall allow patients to view their upcoming and previous appointments. Each appointment shall display the doctor's name, date, time, and status. Patients shall only be able to view appointments associated with their own accounts.

FR-11: Cancel Appointment

The system shall allow patients to cancel eligible appointments. The system shall verify appointment ownership and cancellation eligibility before updating its status. The corresponding availability slot shall be released according to the scheduling rules.

FR-12: Reschedule Appointment

The system shall allow patients to move an eligible appointment to another available slot. The system shall verify the new slot before making changes. If the new slot is unavailable, the original appointment shall remain unchanged.

FR-13: Manage Doctor Availability

The system shall allow doctors to create and update their available appointment slots. The system shall validate dates and times and prevent conflicting availability. A slot containing an existing appointment shall not be modified in a way that invalidates that appointment.

FR-14: View Doctor Appointments

The system shall allow doctors to view appointments assigned to them. The displayed information shall include appointment date, time, status, and relevant patient information. Doctors shall not access appointments assigned to other doctors.

FR-15: Update Appointment Status

The system shall allow doctors to update the status of their assigned appointments. Supported statuses shall include scheduled, confirmed, completed, and cancelled. The system shall validate status changes and reject unauthorized modifications.

FR-16: View Medical History

The system shall allow patients to view their own stored medical history. Authorized doctors shall be able to access relevant patient medical information when permitted by the system. Unauthorized access shall be denied.

FR-17: Create Prescription

The system shall allow doctors to issue prescriptions for authorized patients. A prescription shall contain medication name, dosage, frequency, duration, and optional instructions. The system shall validate required information and associate the prescription with the correct patient and doctor.

FR-18: View Prescriptions

The system shall allow patients to view prescriptions issued to them. Prescription information shall include the issuing doctor, creation date, medication details, dosage, frequency, duration, and additional instructions where provided.

FR-19: Manage User Accounts

The system shall allow administrators to view, search, and manage registered patient and doctor accounts. Permitted actions shall include updating account information and activating or deactivating accounts. Only authenticated administrators may perform these operations.

FR-20: View System User Information

The system shall allow administrators to view an overview of registered users, including names, email addresses, roles, and account statuses. This information shall support basic platform administration.

3.2 Use Case Model

The system has four external actors:

- **Visitor:** Browses public doctor information and registers.
- **Patient:** Books and manages appointments and accesses personal information.
- **Doctor:** Manages availability, appointments, and prescriptions.
- **Administrator:** Manages accounts and user information.

3.2.1 Use Case Diagrams

The following four diagrams must be rendered as images and inserted into the final PDF.

Figure 1 – Patient and Visitor Use Cases

```
@startuml
left to right direction
actor Visitor
actor Patient

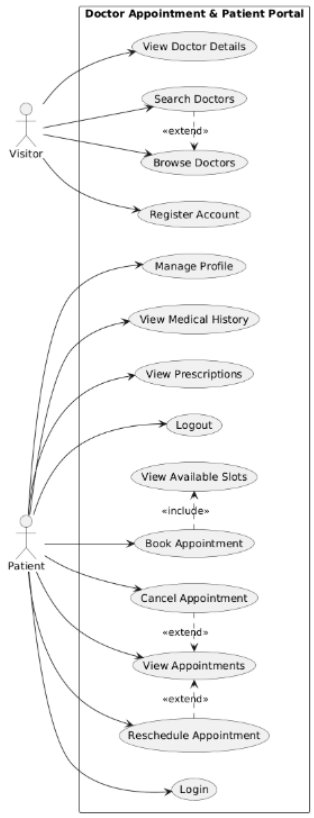
rectangle "Doctor Appointment & Patient Portal" {
  usecase "Register Account" as U1
```

```
  usecase "Login" as U2
  usecase "Browse Doctors" as U3
  usecase "Search Doctors" as U4
  usecase "View Doctor Details" as U5
  usecase "View Available Slots" as U6
  usecase "Book Appointment" as U7
  usecase "View Appointments" as U8
  usecase "Cancel Appointment" as U9
  usecase "Reschedule Appointment" as U10
  usecase "Manage Profile" as U11
  usecase "View Medical History" as U12
  usecase "View Prescriptions" as U13
  usecase "Logout" as U14
}

Visitor --> U1
Visitor --> U3
Visitor --> U4
Visitor --> U5

Patient --> U2
Patient --> U7
Patient --> U8
Patient --> U9
Patient --> U10
Patient --> U11
Patient --> U12
Patient --> U13
Patient --> U14

U7 .> U6 : <<include>>
U4 .> U3 : <<extend>>
U9 .> U8 : <<extend>>
U10 .> U8 : <<extend>>
@enduml
```



Description: Visitors can browse, search, and view public doctor profiles or register. Patients can log in, book and manage appointments, maintain profiles, and access their medical information. Booking includes checking available slots.

Figure 2 – Doctor Use Cases

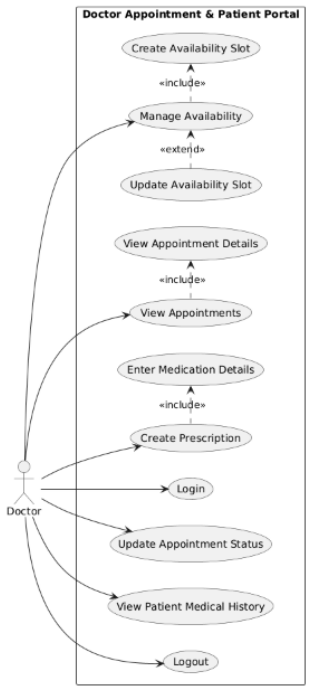
```
@startuml
left to right direction
actor Doctor

rectangle "Doctor Appointment & Patient Portal" {
  usecase "Login" as U1
  usecase "Manage Availability" as U2
  usecase "Create Availability Slot" as U3
  usecase "Update Availability Slot" as U4
  usecase "View Appointments" as U5
  usecase "View Appointment Details" as U6
  usecase "Update Appointment Status" as U7
  usecase "View Patient Medical History" as U8
```

```
  usecase "Create Prescription" as U9
  usecase "Enter Medication Details" as U10
  usecase "Logout" as U11
}

Doctor --> U1
Doctor --> U2
Doctor --> U5
Doctor --> U7
Doctor --> U8
Doctor --> U9
Doctor --> U11

U2 .> U3 : <<include>>
U4 .> U2 : <<extend>>
U5 .> U6 : <<include>>
U9 .> U10 : <<include>>
@enduml
```



Description: Doctors can manage their availability, view assigned appointments, update appointment statuses, access permitted patient information, and issue prescriptions.

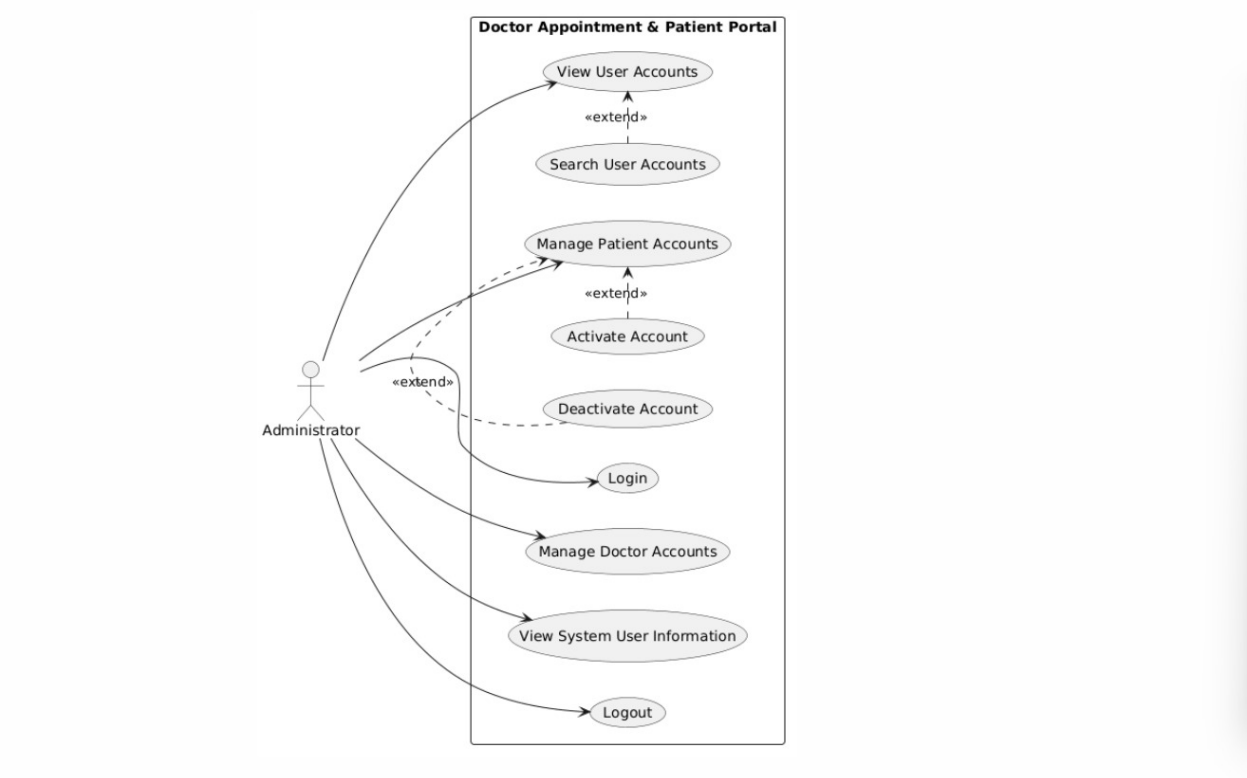
Figure 3 – Administrator Use Cases

```
@startuml
left to right direction
actor Administrator

rectangle "Doctor Appointment & Patient Portal" {
  usecase "Login" as U1
  usecase "View User Accounts" as U2
  usecase "Search User Accounts" as U3
  usecase "Manage Patient Accounts" as U4
  usecase "Manage Doctor Accounts" as U5
  usecase "Activate Account" as U6
  usecase "Deactivate Account" as U7
  usecase "View System User Information" as U8
  usecase "Logout" as U9
}

Administrator --> U1
Administrator --> U2
Administrator --> U4
Administrator --> U5
Administrator --> U8
Administrator --> U9

U3 .> U2 : <<extend>>
U6 .> U4 : <<extend>>
U7 .> U4 : <<extend>>
@enduml
```



Description: Administrators can view and search user accounts, manage patient and doctor accounts, activate or deactivate accounts, and view registered user information.

3.2.2 Use Case Descriptions

UC-01: Register Account

Field	Description
Primary Actor	Visitor
Preconditions	Visitor is not logged in
Trigger	Visitor selects Register
Postconditions	Patient account is created if successful

Main Flow:

1. Visitor opens the registration page.
2. System displays the registration form.
3. Visitor enters required information.
4. System validates the information and checks email uniqueness.
5. System creates the account.
6. System displays confirmation.

Alternative Flow: Missing, invalid, or duplicate information causes an error message and the account is not created.

UC-02: Search and View Doctor

Field	Description
Primary Actor	Visitor or Patient
Preconditions	System is accessible
Trigger	User opens doctor search
Postconditions	Selected public doctor profile is displayed

Main Flow:

1. User opens the doctor search page.
2. User enters a name or specialization.
3. System retrieves matching profiles.
4. User selects a doctor.
5. System displays the doctor's profile.

Alternative Flow: If no matching doctor exists, the system displays a no-results message.

UC-03: Book Appointment

Field	Description
Primary Actor	Patient
Preconditions	Patient is authenticated and a slot is available
Trigger	Patient selects a slot
Postconditions	Appointment is stored and the slot is reserved

Main Flow:

1. Patient selects a doctor.
2. System displays available slots.
3. Patient selects a slot and confirms.
4. System verifies availability.
5. System creates the appointment.
6. System marks the slot as unavailable.
7. System displays confirmation.

Alternative Flow: If the slot is no longer available, the booking is rejected and the patient must select another slot.

UC-04: Manage Availability

Field	Description
Primary Actor	Doctor
Preconditions	Doctor is authenticated
Trigger	Doctor opens availability management
Postconditions	Valid availability changes are saved

Main Flow:

1. Doctor opens the availability page.
2. System displays existing slots.
3. Doctor creates or updates a slot.
4. System validates the date and time.
5. System checks for conflicts and existing appointments.
6. System saves the changes.

Alternative Flow: Conflicting or invalid slots are rejected.

UC-05: Update Appointment Status

Field	Description
Primary Actor	Doctor
Preconditions	Doctor is authenticated and owns the appointment
Trigger	Doctor selects an appointment
Postconditions	Appointment status is updated

Main Flow:

1. Doctor opens the appointment list.
2. Doctor selects an appointment.
3. System displays its details.
4. Doctor selects a new status.
5. System validates and saves the change.
6. System confirms the update.

Alternative Flow: Unauthorized or invalid status changes are rejected.

UC-06: Create Prescription

Field	Description

Primary Actor	Doctor
Preconditions	Doctor is authenticated and authorized to access the patient
Trigger	Doctor selects Create Prescription
Postconditions	Prescription is saved and associated with the patient

Main Flow:

1. Doctor opens the authorized patient record.
2. Doctor selects Create Prescription.
3. System displays the prescription form.
4. Doctor enters medication information.
5. System validates required fields.
6. System saves the prescription.
7. System confirms creation.

Alternative Flow: Missing information or insufficient permissions prevent creation.

UC-07: Manage User Accounts

Field	Description
Primary Actor	Administrator
Preconditions	Administrator is authenticated
Trigger	Administrator opens user management
Postconditions	Valid account changes are saved

Main Flow:

1. Administrator opens the user management page.
2. System displays registered accounts.
3. Administrator searches for or selects an account.
4. Administrator selects a permitted action.
5. System validates the action.
6. System updates the account.
7. System confirms completion.

Alternative Flow: Invalid accounts or unauthorized actions are rejected.

3.3 Domain Model

The domain model describes the main entities and their relationships.

3.3.1 Conceptual Class Diagram

Figure 4 – Conceptual Class Diagram

```
@startuml

abstract class User {
  userId
  fullName
  email
  password
  phoneNumber
  accountStatus
}

class Patient {
  dateOfBirth
  address
}

class Doctor {
  specialization
  professionalDescription
}

class Administrator {
  adminLevel
}

class Appointment {
  appointmentId
  appointmentDate
  startTime
  endTime
  status
}

class AvailabilitySlot {
  slotId
  date
```

```
  startTime
  endTime
  isAvailable
}

class Prescription {
  prescriptionId
  creationDate
  instructions
}

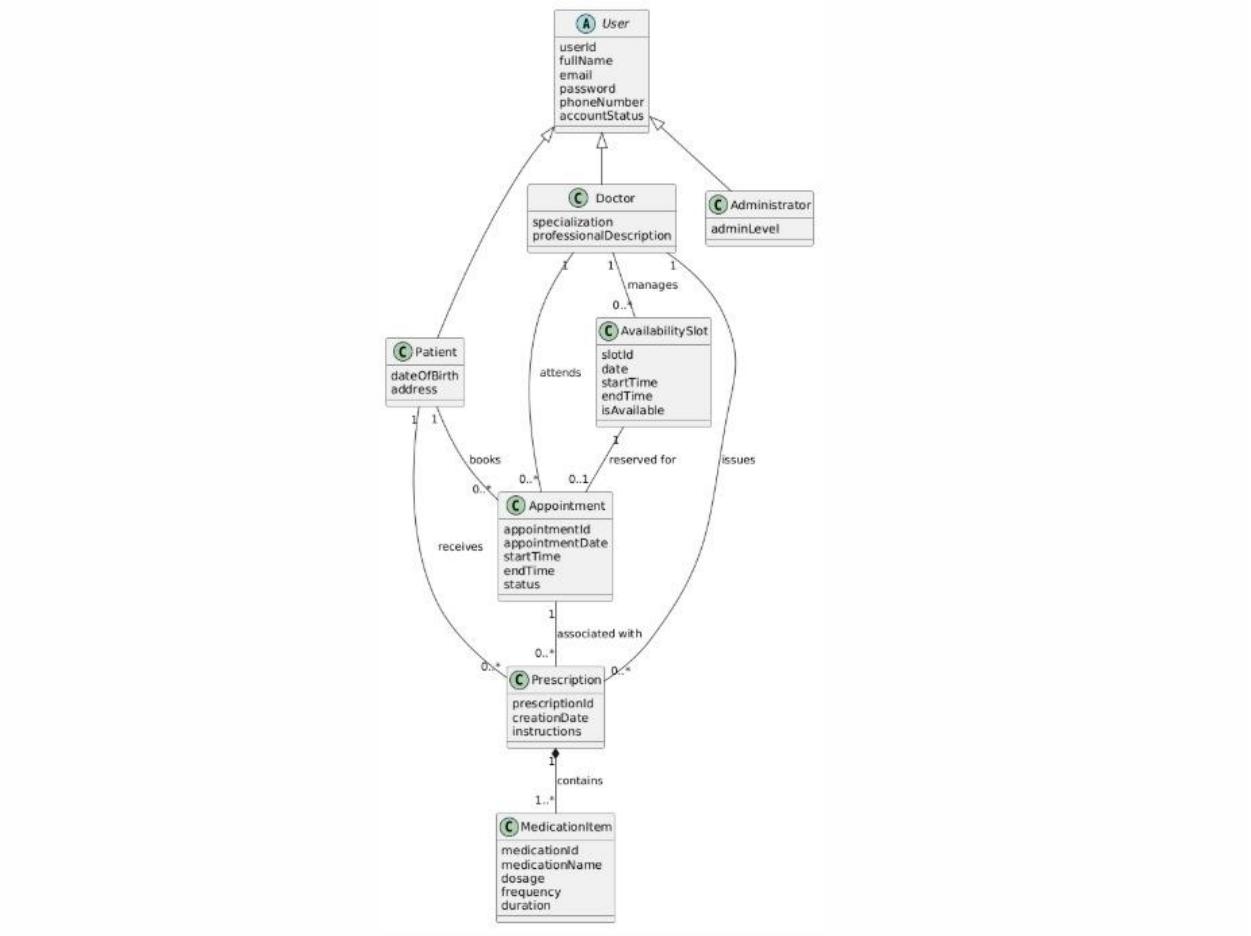
class MedicationItem {
  medicationId
  medicationName
  dosage
  frequency
  duration
}

User <|-- Patient
User <|-- Doctor
User <|-- Administrator

Patient "1" -- "0..*" Appointment : books
Doctor "1" -- "0..*" Appointment : attends
Doctor "1" -- "0..*" AvailabilitySlot : manages
AvailabilitySlot "1" -- "0..1" Appointment : reserved for

Appointment "1" -- "0..*" Prescription : associated with
Patient "1" -- "0..*" Prescription : receives
Doctor "1" -- "0..*" Prescription : issues
Prescription "1" *-- "1..*" MedicationItem : contains

@enduml
```



Description: User is an abstract class containing shared account information. Patient, Doctor, and Administrator inherit from User.

A patient can have multiple appointments, while each appointment belongs to one patient and one doctor. Doctors manage multiple availability slots, and each slot can be reserved by at most one appointment.

Prescriptions are associated with patients and issuing doctors. A prescription contains one or more medication items.

3.3.2 Class Descriptions

Class	Description
User	Common account information shared by all user roles
Patient	Stores patient-specific information
Doctor	Stores doctor specialization and professional details
Administrator	Represents users with administrative permissions
Appointment	Stores consultation date, time, and status

AvailabilitySlot	Stores a doctor's available booking times
Prescription	Stores prescription date and instructions
MedicationItem	Stores individual medication details

Multiplicity Summary:

- One Patient can have zero or many Appointments.
- One Doctor can have zero or many Appointments.
- One Doctor can manage zero or many AvailabilitySlots.
- One AvailabilitySlot can be associated with zero or one Appointment.
- One Prescription contains one or more MedicationItems.

3.4 Non-Functional Requirements

NFR-01: Performance

The system shall respond efficiently under normal usage.

Acceptance Criterion: With up to 50 concurrent users, at least 95% of ordinary page and data-retrieval requests shall complete within three seconds.

Testing: Use a load-testing tool to simulate concurrent users and measure response times.

NFR-02: Security and Access Control

The system shall protect accounts and restrict information according to user roles.

Acceptance Criterion: Patients cannot access other patients' records, doctors cannot access unauthorized appointments, and non-administrators cannot access administrative functions.

Testing: Attempt unauthorized access using accounts with different roles. All unauthorized attempts must be denied.

NFR-03: Booking Reliability

The system shall prevent multiple patients from successfully booking the same slot.

Acceptance Criterion: When 20 simultaneous booking attempts target one available slot, no more than one booking shall succeed.

Testing: Simulate simultaneous booking requests and inspect the resulting appointment records.

NFR-04: Usability

The system shall provide a clear interface for common tasks.

Acceptance Criterion: At least four out of five test participants shall complete patient registration, doctor search, and appointment booking without assistance.

Testing: Conduct a usability test with five participants and record task completion.

NFR-05: Maintainability and Testability

The system shall use a modular MVC structure that separates interface, business logic, and data responsibilities.

Acceptance Criterion: The implementation follows MVC, and unit tests covering at least three core booking rules pass: slot availability, duplicate booking prevention, and appointment ownership validation.

Testing: Review the project structure and execute the relevant unit tests.

3.5 External Interface Requirements

3.5.1 User Interface

The system shall provide role-specific interfaces.

Patient interface:

- Registration and login.
- Doctor search and profiles.
- Appointment booking and management.
- Patient profile.
- Medical history and prescriptions.

Doctor interface:

- Availability management.
- Appointment list and details.
- Patient information access.
- Prescription creation.

Administrator interface:

- User account overview.
- Account search and management.
- System user information.

The interface shall provide clear navigation, input validation, and confirmation messages.

3.5.2 Hardware Interface

No specialized hardware is required. The system shall operate on standard computers, laptops, tablets, and smartphones using their normal display and input capabilities.

3.5.3 Software Interface

The system shall interact with:

- A modern web browser for the user interface.
- An application server for request processing.
- A database management system for storing application data.
- The operating system hosting the client or server environment.

Specific technologies will be determined during implementation.

3.5.4 Communication Interface

The system shall use standard network communication protocols for client-server communication. HTTP or HTTPS shall be supported, with HTTPS recommended for deployment. The application shall return appropriate responses and error messages when requests cannot be completed.

4. Appendices

4.1 Appendix A – Data Dictionary

Data Element	Description	Data Type	Constraint
userId	User identifier	Integer / UUID	Primary key
fullName	User's full name	String	Required
email	User email address	String	Required, unique

password	Stored password representation	String	Required, securely stored
phoneNumber	Contact number	String	As required by registration
accountStatus	Account state	Enum	Active / Inactive
dateOfBirth	Patient birth date	Date	Patient-specific
address	Patient address	String	Patient-specific
specialization	Doctor specialization	String	Required for doctors
appointmentId	Appointment identifier	Integer / UUID	Primary key
appointmentDate	Appointment date	Date	Required
startTime	Appointment start	Time	Required
endTime	Appointment end	Time	Later than start
status	Appointment state	Enum	Scheduled / Confirmed / Completed / Cancelled
slotId	Availability slot identifier	Integer / UUID	Primary key
isAvailable	Slot availability	Boolean	True / False
prescriptionId	Prescription identifier	Integer / UUID	Primary key
creationDate	Prescription creation date	Date	Required
instructions	Prescription instructions	String	Optional
medicationId	Medication item identifier	Integer / UUID	Primary key
medicationName	Medication name	String	Required
dosage	Medication dosage	String	Required
frequency	Medication frequency	String	Required
duration	Medication duration	String	Required

4.2 Appendix B – Glossary

Term	Definition
Authentication	Verification of a user's identity
Authorization	Control of access to system resources
Availability Slot	A time period offered by a doctor for booking
Database	Organized electronic storage of information
Functional Requirement	A description of a service the system must provide
Medical History	Stored medical information associated with a patient
MVC	Model-View-Controller architectural pattern
Non-Functional Requirement	A quality or constraint imposed on the system

Role-Based Access Control	Restricting access according to user roles
Use Case	A description of an actor's interaction with the system
Prescription	Medication instructions issued by a doctor